

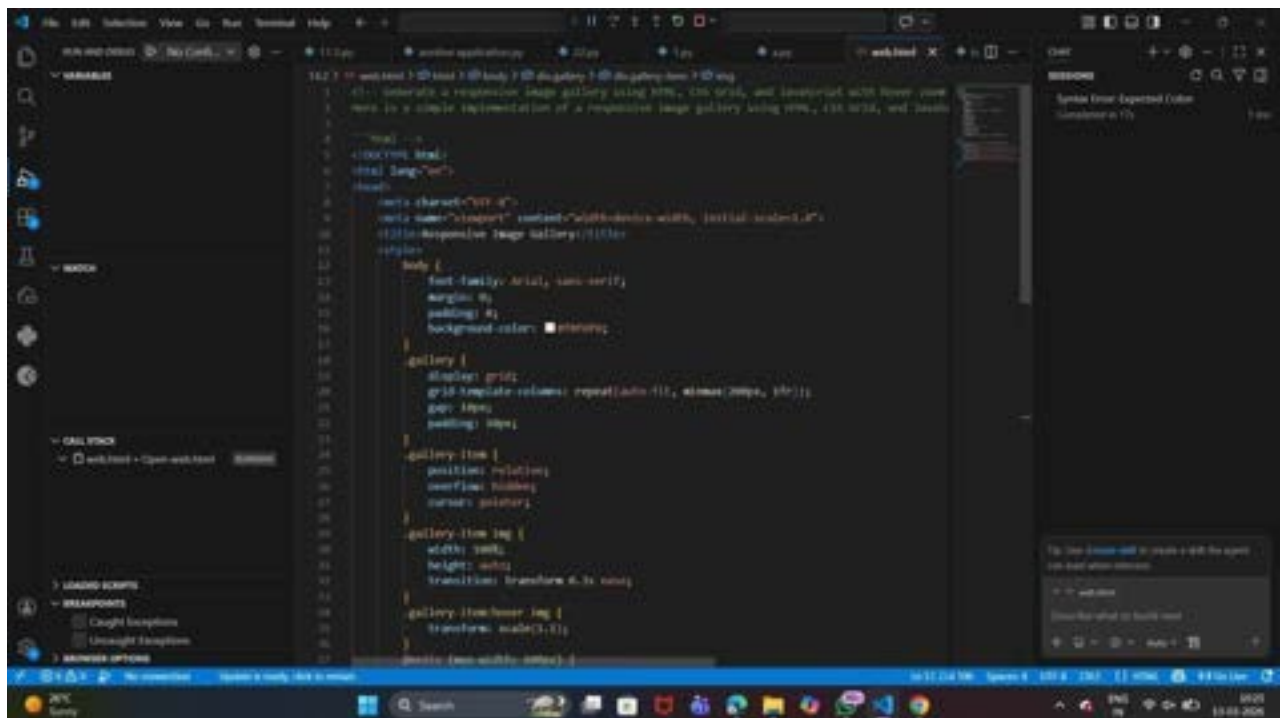
Batch-51

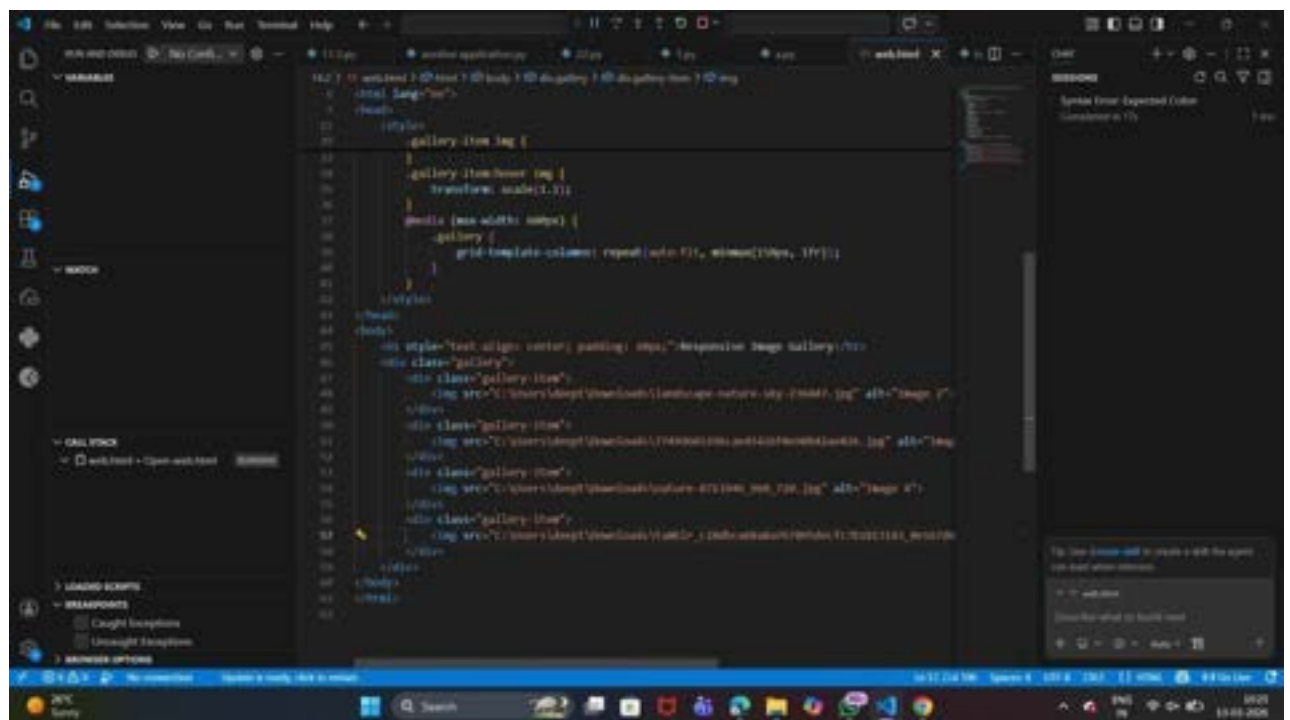
Web Design Application – AI-Assisted HTML/CSS/JS Generation

Task Description #1 (Image Gallery Website):

PROMPT: Generate a responsive image gallery using HTML, CSS Grid, and JavaScript with hover zoom effects and mobile-friendly layout.

CODE:





OUTPUT:



Explanation



Explanation

- Flexbox aligns card elements properly.
- Modern colors and fonts improve UI design.
- Hover animation adds interactive effects.

Task Description #3 (Develop a dynamic shopping cart system):

Prompt : Develop a dynamic shopping cart using HTML, CSS, and JavaScript that allows adding, removing, and updating items with real-time total price and localStorage support.

Code :

```

142 > <html><head><head><style>
1 // Developing a dynamic shopping cart using HTML, CSS, and JavaScript that allows adding, removing,
2 // Manage cart state using JavaScript objects.
3 // Persist cart data in localStorage.
4 // Update total price in real-time as items are added, removed, or updated.
5 // Here is a simple implementation of a dynamic shopping cart using HTML, CSS, and JavaScript (the
6 // HTML)
7 </script></script>
8 <html lang="en">
9 <head>
10 <meta charset="UTF-8">
11 <meta name="viewport" content="width=device-width, initial-scale=1.0">
12 <title>Shopping Cart</title>
13
14 <style>
15 body{
16     font-family: Arial;
17     background-color: #f0f0f0;
18     margin: 20px;
19 }
20
21 .container{
22     max-width: 800px;
23     margin: auto;
24 }
25
26 .product, .cart-item{
27     display: flex;
28     justify-content: space-between;
29     background-color: #fff;
30     padding: 10px;
31     margin: 10px 0;
32     border-radius: 10px;
33 }
34
35 button{
36     padding: 5px 10px;
37     border: none;
38 }
39
40
41
42 </html></html></head></head></style>
43
44 </script>
45
46 </script>
47
48 </body>
49
50 <div class="container">
51 <div>Products</div>
52
53 <div class="product">
54 <span>Phone - $20000</span>
55 <button onclick="addToCart('Phone',20000)">Add</button>
56 </div>
57
58 <div class="product">
59 <span>Laptop - $50000</span>
60 <button onclick="addToCart('Laptop',50000)">Add</button>
61 </div>
62
63 <div class="product">
64 <span>Headphones - $2000</span>
65 <button onclick="addToCart('Headphones',2000)">Add</button>
66 </div>
67
68 <div>View Cart</div>
69 <div id="cart"></div>
70
71 <div class="total">
72 Total: <span id="total">$0</span>
73 </div>
74
75 </div>
76

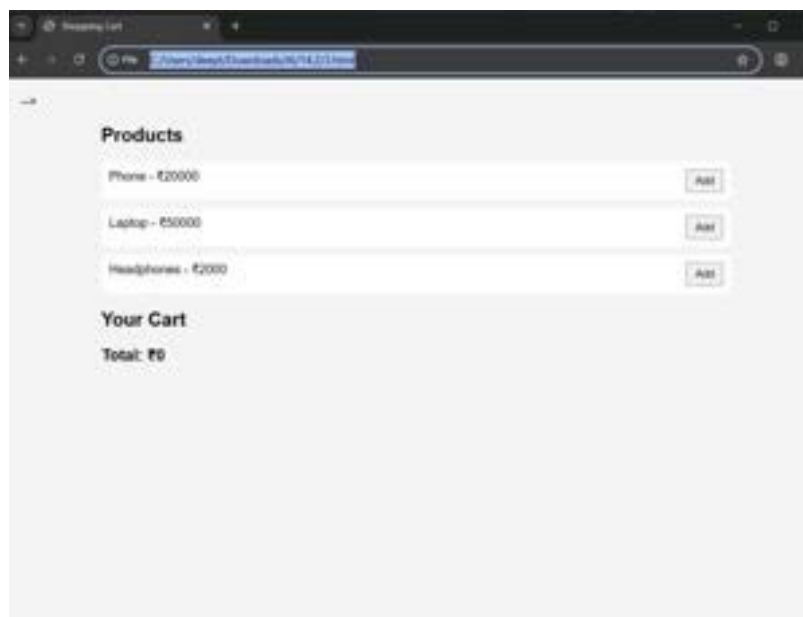
```

```

142 < @html < @head < @style < %body
143 < @div lang="en">
144 < @body>
145 < @script>
146 function updateCart(){
147   let total = 0;
148   for(let item in cart){
149     let price = cart[item].price;
150     let qty = cart[item].qty;
151     total += price * qty;
152   }
153   cartDiv.innerHTML += `
154   <div class="cart-item">
155     <span>{item}</span> - <span>{price}</span>
156     <input type="button" value="{qty}" size="1"
157     onchange="changeQty('{item}',this.value)"/>
158     <button onclick="removeItem('{item}')">Remove</button>
159   </div>
160   `;
161   document.getElementById("total").innerText = total;
162   localStorage.setItem("cart", JSON.stringify(cart));
163 }
164 updateCart();
165 </script>
166 </div>
167 </body>
168 </html>

```

Output :



Explanation

- JavaScript objects manage cart items.
- Total price updates automatically.
- localStorage saves cart data after refresh.

Task description #4 (Role-Based Access Control Application):

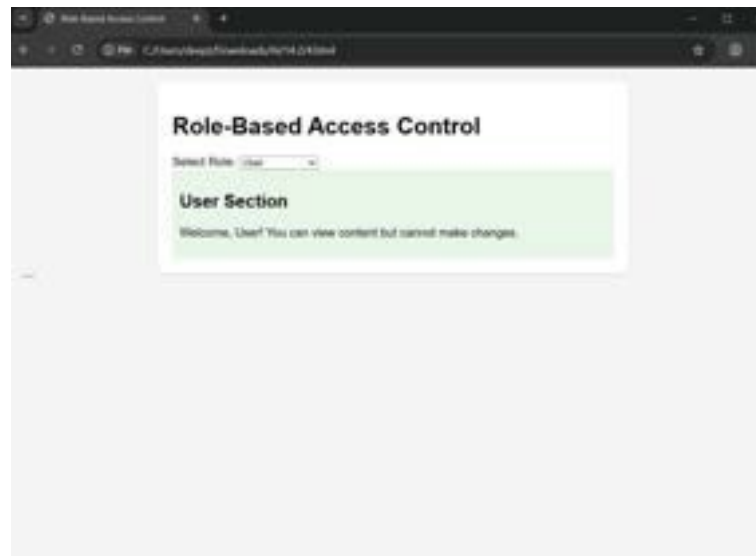
Prompt : Create a role-based web application using HTML, CSS, and JavaScript with Admin, Editor, and User roles and conditional UI rendering and Design access control logic, Maintain secure state transitions, Create scalable UI components.

Code :

```
142 > -> 4.html >
1  <!-- Create a Role-Based Access Control web app using HTML, CSS, and JavaScript.
2  Roles: Admin, Editor, User.
3  Show different UI sections based on role using conditional rendering.
4  Here is a simple implementation of a Role-Based Access Control (RBAC) web app using HTML, CSS,
5  and JavaScript. -->
6  <!DOCTYPE html>
7  <html lang="en">
8  <head>
9      <meta charset="UTF-8">
10     <meta name="viewport" content="width=device-width, initial-scale=1.0">
11     <title>Role-Based Access Control</title>
12     <style>
13         body {
14             font-family: Arial, sans-serif;
15             background-color: #f0f0f0;
16             margin: 0;
17             padding: 20px;
18         }
19         .container {
20             max-width: 600px;
21             margin: auto;
22             background-color: #fff;
23             padding: 20px;
24             border-radius: 8px;
25             box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
26         }
27         .section {
28             display: none;
29         }
30         .admin-section {
31             background-color: #e0f2f1;
32             padding: 10px;
33             margin-bottom: 10px;
34         }
35         .editor-section {
36             background-color: #e0f2f1;
37             padding: 10px;
38         }
39     </style>
40 </head>
```

```
142 > -> 4.html >
41 <body>
42     <div class="container">
43         <div id="role" onchange="renderSections()">
44             <option value="">Select Role</option>
45             <option value="admin">Admin</option>
46             <option value="editor">Editor</option>
47             <option value="user">User</option>
48         </div>
49
50         <div class="section admin-section" id="adminSection">
51             <div>Admin Section</div>
52             <p>Welcome, Admin! You have full access to the system.</p>
53         </div>
54
55         <div class="section editor-section" id="editorSection">
56             <div>Editor Section</div>
57             <p>Welcome, Editor! You can edit content but have limited access to settings.</p>
58         </div>
59
60         <div class="section user-section" id="userSection">
61             <div>User Section</div>
62             <p>Welcome, User! You can view content but cannot make changes.</p>
63         </div>
64     </div>
65
66     <script>
67         function renderSections() {
68             const role = document.getElementById('role').value;
69             document.getElementById('adminSection').style.display = role === 'admin' ? 'block' : 'none';
70             document.getElementById('editorSection').style.display = role === 'editor' ? 'block' : 'none';
71             document.getElementById('userSection').style.display = role === 'user' ? 'block' : 'none';
72         }
73     </script>
74 </body>
75 </html>
```

Output :



Explanation

- Different roles have different permissions.
- UI changes based on user role.
- Improves security and access control.

Task Description #5 (Multi-Language Website-Design a multilingual website):

Prompt : Design a multilingual website using HTML, CSS, and JavaScript with dynamic language switching and language preference stored in localStorage and Structure translation files,Handle RTL/LTR layouts,Improve accessibility.

Code :


```

142 > @html > @head > @style > % container
1  <!-- Devlop a multilingual website using HTML, CSS, and Javascript.
2  Allow dynamic language switching.
3  Store translations in JSON and save language preference in localStorage.
4  Here is a simple implementation of a multilingual website using HTML, CSS, and Javascript that
5  -- HTML -->
6  <doctype html>
7  <html lang="en">
8  <head>
9      <meta charset="UTF-8">
10     <meta name="viewport" content="width=device-width, initial-scale=1.0">
11     <title>multilingual website</title>
12     <style>
13         body {
14             font-family: arial, sans-serif;
15             background-color: #e0e0e0;
16             margin: 0;
17             padding: 20px;
18         }
19         .container {
20             max-width: 600px;
21             margin: auto;
22             background-color: #ffffff;
23             padding: 20px;
24             border-radius: 4px;
25             box-shadow: 0 2px 4px #888;
26         }
27         .language-selector {
28             margin-bottom: 20px;
29         }
30     </style>
31 </head>
32 <body>
33     <div class="container">
34         <div id="title">welcome to our website</div>
35         <div id="description">This is a simple multilingual website example.</div>
36         <div class="language-selector">
37             <select><option value="en">English</option></select>

```

```

142 > @html > @head > @style > % container
7  <html lang="en">
32 <body>
33     <div class="container">
34     </div>
35
36     <script>
37         const translations = {
38             en: {
39                 title: "welcome to our website!",
40                 description: "This is a simple multilingual website example."
41             },
42             es: {
43                 title: "¡Bienvenido a nuestro sitio web!",
44                 description: "Este es un ejemplo de sitio web multilingüe simple."
45             },
46             fr: {
47                 title: "Bienvenue sur notre site web!",
48                 description: "Ceci est un exemple de site web multilingue simple."
49             }
50         };
51
52         function switchLanguage() {
53             const language = document.getElementById("language").value;
54             document.getElementById("title").textContent = translations[language].title;
55             document.getElementById("description").textContent = translations[language].description;
56             localStorage.setItem("preferredLanguage", language);
57         }
58
59         // load preferred language on page load
60         window.onload = function() {
61             const preferredLanguage = localStorage.getItem("preferredLanguage") || "en";
62             document.getElementById("language").value = preferredLanguage;
63             switchLanguage();
64         };
65     </script>
66 </body>
67 </html>

```

Output :



Explanation:

- JSON files store translation data.
- Users can switch languages dynamically.
- Language preference is saved for future visits.

Task Description #6 (Online Examination System-Create an online exam interface.):

Prompt : Create an online examination interface using HTML, CSS, and JavaScript with MCQ questions, countdown timer, and single submission system and Manage exam state, Improve exam UX, Ensure accessibility.

Code :

```
<div></div></body></html>
```

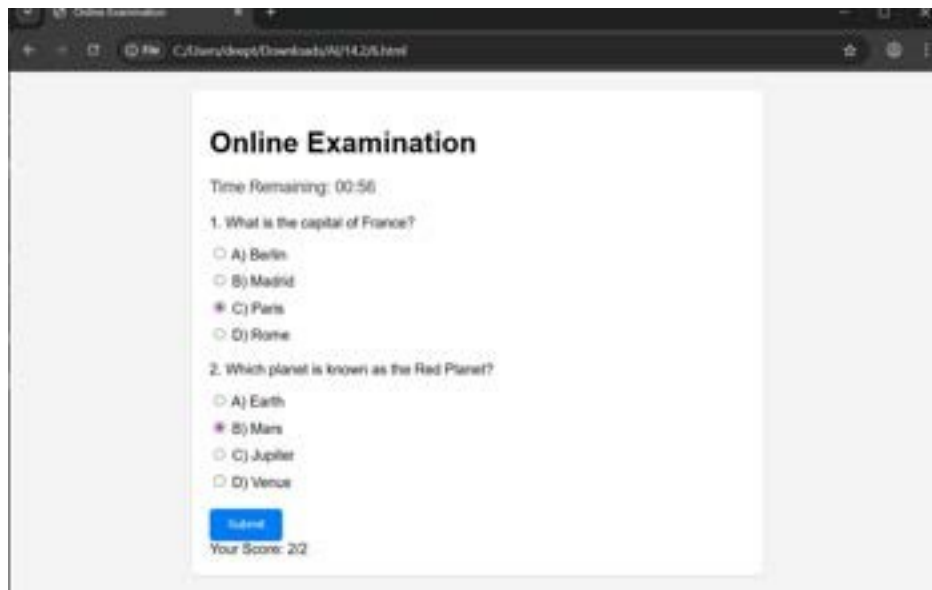
```
06: // class > @size > @text > @style > @content  
7      <div lang="en">  
8          <p>  
9              <br/>  
10             </styles>  
11         </head>  
12         <body>  
13             <div class="container">  
14                 <div id="time">Time Remaining: @time</div>  
15                 <div class="question">  
16                     <p>q1, What is the capital of France?</p>  
17                     <ul style="list-style-type: none;">  
18                         <li><input type="radio" name="q1" value="A"/> A. Berlin</li>  
19                         <li><input type="radio" name="q1" value="B"/> B. Madrid</li>  
20                         <li><input type="radio" name="q1" value="C"/> C. Paris</li>  
21                         <li><input type="radio" name="q1" value="D"/> D. Rome</li>  
22                     </ul>  
23                 </div>  
24                 <div class="question">  
25                     <p>q2, Which planet is known as the Red Planet?</p>  
26                     <ul style="list-style-type: none;">  
27                         <li><input type="radio" name="q2" value="A"/> A. Earth</li>  
28                         <li><input type="radio" name="q2" value="B"/> B. Mars</li>  
29                         <li><input type="radio" name="q2" value="C"/> C. Jupiter</li>  
30                         <li><input type="radio" name="q2" value="D"/> D. Venus</li>  
31                     </ul>  
32                 </div>  
33                 <div class="submit_btn" id="submitButton">Submit</div>  
34             </div>  
35             <div id="result"></div>  
36         </script>  
37         let timerLeft = 60;  
38         const timeElement = document.getElementById('time');  
39         const submitBtn = document.getElementById('submitButton');  
40         const resultElement = document.getElementById('result');  
41         let timerInterval;
```

```

1 <html>
2 <head>
3 <title> Timer </title>
4 <script>
5     function startTimer() {
6         timerInterval = setInterval(() => {
7             submitExam();
8         }, 1000);
9     }
10
11     function submitExam() {
12         submitBtn.disabled = true;
13         const q1Answer = document.querySelector('input[name="q1"]:checked').value;
14         const q2Answer = document.querySelector('input[name="q2"]:checked').value;
15         let score = 0;
16
17         if (q1Answer === 'C') score++;
18         if (q2Answer === 'B') score++;
19
20         resultElement.textContent = `Your score: ${score}/2`;
21     }
22
23     submitBtn.addEventListener('click', () => {
24         clearInterval(timerInterval);
25         submitExam();
26     });
27
28     startTimer();
29 </script>
30 </body>
31 </html>

```

Output :



The screenshot shows a web browser window with the address bar displaying "C:/Users/deep/Downloads/N/14.2/5.html". The page content is titled "Online Examination". Below the title, it says "Time Remaining: 00:56". There are two questions:

1. What is the capital of France?

- ☐ A) Berlin
- ☐ B) Madrid
- ☒ C) Paris
- ☐ D) Rome

2. Which planet is known as the Red Planet?

- ☐ A) Earth
- ☒ B) Mars
- ☐ C) Jupiter
- ☐ D) Venus

At the bottom, there is a blue "Submit" button and the text "Your Score: 2/2".

Explanation :

- Timer controls exam duration.
- JavaScript manages exam state.
- Prevents multiple submissions.